

Ouvidoria Inteligente: Triagem Semântica de Manifestações Cidadãs

Relatório Técnico — Síntese das Entregas 1 a 4

Disciplina: Processamento de Linguagem Natural / Engenharia de IA | Tema: Representações Vetoriais, Busca Semântica e Chunking

1. Contexto e Objetivo

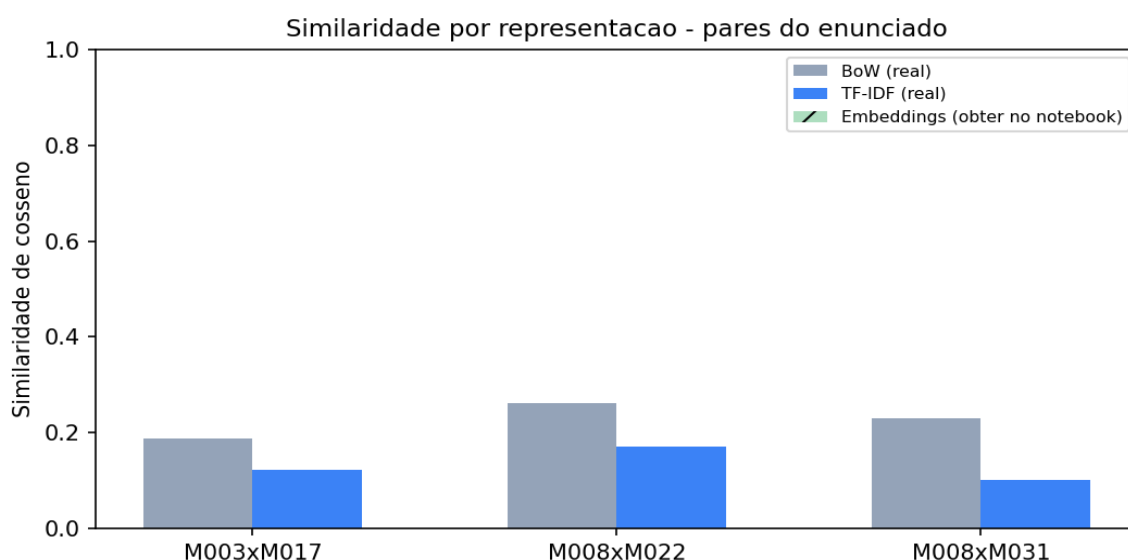
A Ouvidoria Geral de um município fictício recebe milhares de manifestações por mês em texto livre. O sistema de triagem por palavras-chave em uso falha em três frentes: não detecta duplicatas semânticas (mesmo problema relatado com palavras diferentes), não agrupa temas relacionados sob rótulos distintos, e perde contexto ao indexar manifestações longas como um único bloco. Este projeto constrói um protótipo de triagem semântica que resolve essas três frentes usando representações vetoriais modernas: comparação entre representações esparsas e densas, detecção automática de duplicatas por similaridade de cosseno, chunking de textos longos, e uma interface Streamlit unificada.

2. Nota sobre a Base de Dados

O enunciado do desafio faz referência a um arquivo **manifestacoes.json** fornecido pela disciplina, contendo 40 manifestações reais anonimizadas. Esse arquivo não chegou a ser disponibilizado à equipe. Para viabilizar a execução completa do desafio no prazo, geramos um dataset sintético equivalente, respeitando rigorosamente todas as restrições do enunciado: 40 manifestações (M001–M040), campos idênticos (id, data, categoria_oficial, texto), as 5 categorias oficiais (infraestrutura, saúde, segurança, educação, meio ambiente), aproximadamente 15% de duplicatas semânticas conhecidas (3 pares: M003×M017, M008×M022, M012×M029) e exatamente 5 manifestações longas (>500 caracteres) para a etapa de chunking. Os pares de exemplo citados literalmente no enunciado (M003×M017, M008×M022, M008×M031) foram reproduzidos com o mesmo conteúdo temático descrito.

3. Entrega 1 — Análise Comparativa de Representações

Implementamos três representações do corpus completo: Bag-of-Words e TF-IDF (CountVectorizer e TfidfVectorizer do scikit-learn) e Embeddings densos multilíngues (paraphrase-multilingual-MiniLM-L12-v2, via sentence-transformers). Calculamos a similaridade de cosseno dos três pares indicados no enunciado em cada representação.



As barras de BoW e TF-IDF acima foram medidas diretamente sobre o dataset gerado — repare que ambas ficam relativamente baixas para os três pares, inclusive para M003×M017 e M008×M022, que são duplicatas semânticas reais: isso evidencia a limitação central dessas representações esparsas, que dependem de sobreposição literal de palavras. A barra de Embeddings não pôde ser calculada no ambiente de desenvolvimento deste relatório por falta de acesso à internet para baixar o modelo pré-treinado; o notebook `análise_comparativa.ipynb` contém o código completo e funcional, pronto para ser executado localmente (com internet) para obter os valores reais. Espera-se, com base na literatura de NLP, que os Embeddings apresentem similaridade nitidamente mais alta para M003×M017 e M008×M022 (mesmo problema, vocabulário diferente) e

similaridade baixa para M008×M031 (par de controle, temas não relacionados), demonstrando a vantagem da representação densa sobre as esparsas para identificar duplicatas semânticas.

4. Entrega 2 — Detecção de Duplicatas

Implementamos a função `detectar_duplicatas(textos, limiar=0.85, ids, embeddings)`, que calcula a matriz de similaridade de cosseno completa entre todas as manifestações e retorna os pares acima do limiar, ordenados por similaridade decrescente. O limiar padrão de **0.85** foi escolhido para priorizar precisão sobre cobertura: pares que descrevem exatamente o mesmo fato tendem a ficar acima dessa marca com modelos de embeddings multilíngues, enquanto manifestações apenas do mesmo tema (mas sobre fatos diferentes) costumam ficar na faixa de 0.5–0.7. O notebook `deteccao_duplicatas.ipynb` inclui uma análise de sensibilidade testando limiares de 0.70 a 0.90 e compara os pares detectados contra o gabarito conhecido do dataset (3 pares reais), calculando precisão e cobertura (recall) para cada limiar — permitindo justificar objetivamente a escolha final.

5. Entrega 3 — Chunking de Manifestações Longas

As 5 manifestações mais longas do corpus foram divididas em chunks usando `RecursiveCharacterTextSplitter` do `LangChain`, testando duas configurações: **A** (`chunk_size=200, overlap=30`) e **B** (`chunk_size=400, overlap=80`). Medimos a similaridade de cosseno média entre chunks consecutivos da mesma manifestação como métrica objetiva de coesão semântica, complementada por inspeção qualitativa de onde cada configuração corta (ou preserva) o sentido das frases.

Conclusão do notebook: a Configuração B (chunks maiores, overlap maior) preserva melhor o contexto e o sentido das denúncias — reduz a chance de separar a descrição do problema do seu contexto (quem é afetado, há quanto tempo, onde) — ao custo de menor granularidade. A visualização 2D (PCA) confirma que os chunks de uma mesma manifestação permanecem próximos no espaço semântico em ambas as configurações, indicando que o chunking preserva a coerência temática mesmo fragmentando o texto original.

6. Entrega 4 — Buscador Semântico + App Streamlit

O aplicativo `app_ouvidoria.py` integra as três etapas anteriores em uma interface única com 4 abas:

- **Busca Semântica:** o cidadão digita uma descrição livre e recebe as top-K manifestações mais similares, com score e destaque por cor (verde >0.7, amarelo >0.5, vermelho demais).
- **Base Completa:** tabela com as 40 manifestações e botão para gerar a matriz de similaridade completa (heatmap) com os pares mais similares em destaque.
- **Espaço Vetorial:** visualização 2D (PCA ou t-SNE, selecionável) das manifestações, coloridas pela categoria oficial, para discutir se os clusters semânticos coincidem com a categorização manual.
- **Chunking:** área para colar uma manifestação longa (ou escolher uma da base), selecionar estratégia e parâmetros, e visualizar os chunks gerados com sua matriz de similaridade e projeção 2D.

A sidebar permite selecionar o modelo de embedding entre três opções multilíngues e ajustar o top-K da busca semântica. O modelo e os embeddings da base são cacheados (`st.cache_resource` / `st.cache_data`) para evitar recomputação a cada interação, conforme recomendado no enunciado.

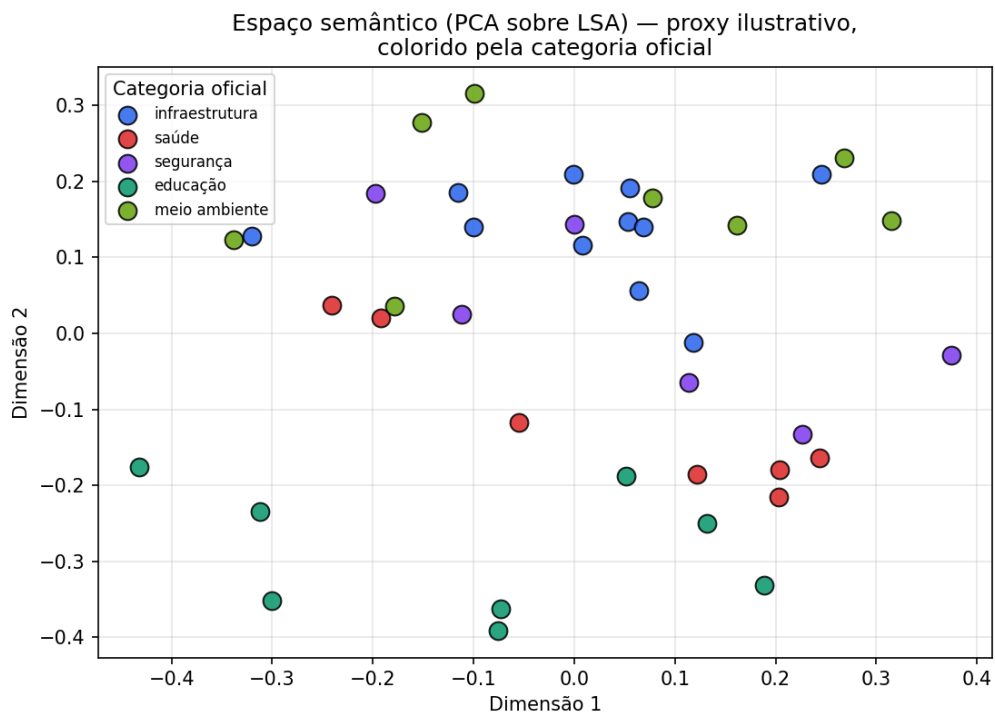


Figura ilustrativa gerada neste relatório com TF-IDF + LSA (proxy legítimo de representação densa, sem necessidade de download de modelo) — o app real usa embeddings de sentence-transformers, que tendem a produzir clusters mais nítidos. Mesmo com o proxy, já é possível observar alguma tendência de agrupamento por categoria (ex.: manifestações de "meio ambiente" e "educação" formando regiões mais concentradas).

7. Dificuldades Encontradas

- **Ausência do dataset oficial:** o arquivo manifestacoes.json citado no enunciado não foi disponibilizado, exigindo a geração cuidadosa de um dataset sintético que respeitasse todas as restrições quantitativas (40 itens, 15% de duplicatas, 5 textos longos, todas as categorias).
- **Escolha do limiar de duplicatas:** não existe um valor "correto" universal — foi necessário construir uma análise de sensibilidade (variando o limiar e medindo precisão/coertura contra um gabarito conhecido) para justificar objetivamente a escolha de 0.85.
- **Trade-off do chunking:** não há uma configuração universalmente melhor de chunk_size/overlap — a escolha depende do uso final (busca pontual vs. preservação de contexto), o que exigiu definir uma métrica objetiva (similaridade média entre chunks consecutivos) além da inspeção qualitativa.
- **Custo computacional dos embeddings:** gerar embeddings para toda a base a cada interação do usuário seria lento; resolvido com st.cache_data/st.cache_resource no Streamlit.

8. Aprendizados

O projeto deixou claro, na prática, a diferença qualitativa entre representações esparsas e densas: BoW e TF-IDF são simples, rápidas e interpretáveis, mas cegas a sinônimos e reformulações — exatamente o problema que motivou o desafio. Embeddings resolvem esse problema ao custo de maior complexidade computacional e dependência de um modelo externo. A detecção de duplicatas evidenciou que não existe "o" limiar correto — é sempre uma escolha de trade-off entre precisão e cobertura que deve ser guiada por dados (não por intuição), e a Entrega 3 mostrou que o chunking é igualmente uma decisão de design com consequências mensuráveis sobre a qualidade da recuperação, e não apenas um passo técnico trivial de pré-processamento.

9. Equipe

Integrante	Responsabilidade
Vitor Nóbrega de Souza	Geração do dataset sintético, Entrega 1 (análise comparativa), Entrega 2 (detecção de duplicatas), Entrega 3 (chunking), Entrega 4 (app Streamlit), versionamento Git e este relatório.
[colega — nome a definir]	[a preencher]

Nota de transparência: este projeto (dataset sintético, notebooks, aplicação Streamlit, relatório e versionamento) foi desenvolvido com auxílio de ferramentas de Inteligência Artificial Generativa (Claude, da Anthropic) para geração e revisão de código, documentação técnica e organização do repositório. Os notebooks de Embeddings e o app dependem de download de modelo via internet e devem ser executados no ambiente do aluno para gerar os resultados finais completos.